

# OpenMP Lab Manual Part 2 - Work Sharing Directives

## Lab Overview

This lab focuses on OpenMP work sharing directives, particularly the `for` directive, loop scheduling strategies, and the `single` directive. Students will learn to efficiently distribute loop iterations among threads, understand different scheduling policies, and control single-threaded execution within parallel regions.

## Lab Objectives

By the end of this lab, students will be able to:

1. Use the `#pragma omp for` directive to parallelize loops
2. Understand and identify loop-carried dependencies
3. Apply different loop scheduling strategies (static, dynamic, guided)
4. Use the `single` directive for single-threaded execution
5. Implement parallel algorithms for  $\pi$  estimation, bubble sort, and matrix-vector multiplication
6. Analyze performance differences between scheduling strategies

## Prerequisites

- Completion of OpenMP Lab Manual Part 1
- Understanding of basic parallel regions and thread management
- Knowledge of reduction operations and synchronization
- Basic understanding of algorithms and data structures

---

## Part 1: Work Sharing Directives

### 1.1 The `for` Directive

Work sharing directives appear inside a parallel region and indicate how work should be shared among threads. The most common work sharing directive is the `for` directive.

#### Syntax:

```
cpp
```

```
#pragma omp for [clauses]
```

```
for loop
```

## Key Requirements:

- Must appear inside a parallel region
- Loop must have determinable trip count
- Loop variable must be integer or pointer type
- Loop bounds must not change during execution

## Example 1.1: Basic Parallel For

```
cpp
```

```
#include <iostream>
```

```
#include <omp.h>
```

```
int main() {
```

```
    const int n = 16;
```

```
    int arr[n];
```

```
    std::cout << "=== Method 1: Separate parallel and for directives ===" << std::endl;
```

```
    #pragma omp parallel num_threads(4)
```

```
    {
```

```
        #pragma omp for
```

```
        for (int i = 0; i < n; i++) {
```

```
            arr[i] = 2 * i + 1;
```

```
            std::cout << "Thread " << omp_get_thread_num()
```

```
                        << " processing i = " << i
```

```
                        << ", arr[" << i << "] = " << arr[i] << std::endl;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

## Example 1.2: Combined Parallel For

```
cpp
```

```

#include <iostream>
#include <omp.h>

int main() {
    const int n = 16;
    int arr[n];

    std::cout << "=== Method 2: Combined parallel for directive ===" << std::endl;
    #pragma omp parallel for num_threads(4)
    for (int i = 0; i < n; i++) {
        arr[i] = i * i;
        std::cout << "Thread " << omp_get_thread_num()
                  << " processing i = " << i
                  << ", arr[" << i << "] = " << arr[i] << std::endl;
    }

    return 0;
}

```

### Exercise 1.1:

1. Run both examples and observe how iterations are distributed among threads
2. Change the number of threads to 2 and 8, observe the distribution
3. Explain why both methods produce the same result

## 1.2 Loop-Carried Dependencies

Not all loops can be parallelized safely. A loop-carried dependency occurs when a memory location is read or written in one iteration and written in another iteration.

### Example 1.3: Fibonacci Sequence - Incorrect Parallelization

cpp

```

#include <iostream>
#include <omp.h>

int main() {
    const int n = 10;
    long fibo[n];

    // Serial version (correct)
    std::cout << "=== Serial Fibonacci ===" << std::endl;
    fibo[0] = fibo[1] = 1;
    for (int i = 2; i < n; i++) {
        fibo[i] = fibo[i-1] + fibo[i-2];
    }

    std::cout << "Serial result: ";
    for (int i = 0; i < n; i++) {
        std::cout << fibo[i] << " ";
    }
    std::cout << std::endl;

    // Parallel version (incorrect - has loop-carried dependency)
    std::cout << "=== Parallel Fibonacci (INCORRECT) ===" << std::endl;
    fibo[0] = fibo[1] = 1;
    #pragma omp parallel for num_threads(4)
    for (int i = 2; i < n; i++) {
        fibo[i] = fibo[i-1] + fibo[i-2];
        std::cout << "Thread " << omp_get_thread_num()
            << " computed fibo[" << i << "] = " << fibo[i] << std::endl;
    }

    std::cout << "Parallel result: ";
    for (int i = 0; i < n; i++) {
        std::cout << fibo[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}

```

## Exercise 1.2:

1. Run the Fibonacci example multiple times

2. Explain why the parallel version produces incorrect results
  3. Identify the loop-carried dependency in the code
- 

## Part 2: Tutorial 1 - Estimating $\pi$ (Pi)

### 2.1 Mathematical Background

We can estimate  $\pi$  using the integral:

$$\pi = 4 \times \int_0^1 \sqrt{1-x^2} \, dx$$

This represents the area under a quarter circle with radius 1.

#### Example 2.1: Serial $\pi$ Estimation

```
cpp
```

```

#include <iostream>
#include <chrono>
#include <cmath>

int main() {
    long n = 100000000; // Number of intervals
    double h = 1.0 / n;
    double sum = 0.0;

    auto start = std::chrono::high_resolution_clock::now();

    for (long i = 0; i < n; i++) {
        double x = (i + 0.5) * h;
        sum += sqrt(1.0 - x * x);
    }

    double pi_estimate = 4.0 * h * sum;

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Serial  $\pi$  estimation:" << std::endl;
    std::cout << "Estimated  $\pi$  = " << pi_estimate << std::endl;
    std::cout << "Actual  $\pi$  = " << M_PI << std::endl;
    std::cout << "Error = " << fabs(pi_estimate - M_PI) << std::endl;
    std::cout << "Time = " << duration.count() << " ms" << std::endl;

    return 0;
}

```

## 2.2 Parallel $\pi$ Estimation - Method 1 (Loop Dependency Issue)

### Example 2.2: Incorrect Parallel Implementation

cpp

```

#include <iostream>
#include <omp.h>
#include <chrono>
#include <cmath>

int main() {
    long n = 100000000;
    double h = 1.0 / n;
    double sum = 0.0;

    auto start = std::chrono::high_resolution_clock::now();

    // INCORRECT: This has a loop-carried dependency on 'sum'
    #pragma omp parallel for num_threads(4)
    for (long i = 0; i < n; i++) {
        double x = (i + 0.5) * h;
        sum += sqrt(1.0 - x * x); // Race condition!
    }

    double pi_estimate = 4.0 * h * sum;

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Parallel  $\pi$  estimation (INCORRECT):" << std::endl;
    std::cout << "Estimated  $\pi$  = " << pi_estimate << std::endl;
    std::cout << "Actual  $\pi$  = " << M_PI << std::endl;
    std::cout << "Error = " << fabs(pi_estimate - M_PI) << std::endl;
    std::cout << "Time = " << duration.count() << " ms" << std::endl;

    return 0;
}

```

## 2.3 Parallel $\pi$ Estimation - Method 2 (Using Reduction)

### Example 2.3: Correct Parallel Implementation with Reduction

cpp

```

#include <iostream>
#include <omp.h>
#include <chrono>
#include <cmath>

int main() {
    long n = 100000000;
    double h = 1.0 / n;
    double sum = 0.0;

    auto start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel for num_threads(4) reduction(+:sum)
    for (long i = 0; i < n; i++) {
        double x = (i + 0.5) * h;
        sum += sqrt(1.0 - x * x);
    }

    double pi_estimate = 4.0 * h * sum;

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Parallel  $\pi$  estimation (CORRECT):" << std::endl;
    std::cout << "Estimated  $\pi$  = " << pi_estimate << std::endl;
    std::cout << "Actual  $\pi$  = " << M_PI << std::endl;
    std::cout << "Error = " << fabs(pi_estimate - M_PI) << std::endl;
    std::cout << "Time = " << duration.count() << " ms" << std::endl;

    return 0;
}

```

## 2.4 Parallel $\pi$ Estimation - Method 3 (Private Variables)

### Example 2.4: Using Private Variables and Critical Section

cpp



```

#include <iostream>
#include <omp.h>
#include <chrono>
#include <cmath>

int main() {
    long n = 100000000;
    double h = 1.0 / n;
    double sum = 0.0;

    auto start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel num_threads(4) default(none) shared(n, h, sum)
    {
        double factor = 0.0; // Private variable

        #pragma omp for
        for (long i = 0; i < n; i++) {
            double x = (i + 0.5) * h;
            factor += sqrt(1.0 - x * x);
        }

        #pragma omp critical
        sum += factor;
    }

    double pi_estimate = 4.0 * h * sum;

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Parallel  $\pi$  estimation (Private + Critical):" << std::endl;
    std::cout << "Estimated  $\pi$  = " << pi_estimate << std::endl;
    std::cout << "Actual  $\pi$  = " << M_PI << std::endl;
    std::cout << "Error = " << fabs(pi_estimate - M_PI) << std::endl;
    std::cout << "Time = " << duration.count() << " ms" << std::endl;

    return 0;
}

```

## Exercise 2.1:

1. Implement and run all three versions of  $\pi$  estimation

2. Compare the accuracy and performance of each method
  3. Run the incorrect version multiple times - do you get consistent results?
  4. Test with different numbers of threads (2, 4, 8) and record performance
- 

## Part 3: Loop Scheduling

### 3.1 Understanding Loop Scheduling

OpenMP provides different strategies for distributing loop iterations among threads. The choice of scheduling can significantly impact performance, especially when iterations have different computational loads.

#### Scheduling Types:

- **Static:** Iterations assigned before loop execution
- **Dynamic:** Iterations assigned during execution as threads become available
- **Guided:** Like dynamic, but chunk size decreases over time
- **Runtime:** Schedule determined by environment variable

#### Example 3.1: Comparing Different Schedules

```
cpp
```

```

#include <iostream>
#include <omp.h>
#include <chrono>
#include <cmath>

// Function with varying computational load
double compute_function(int i) {
    double result = 0.0;
    // More work for larger i
    for (int j = 0; j < i; j++) {
        result += sin(j * 0.1);
    }
    return result;
}

void test_schedule(const char* schedule_name, int n, int num_threads) {
    std::cout << "\n=== Testing " << schedule_name << " Schedule ===" << std::endl;

    auto start = std::chrono::high_resolution_clock::now();

    if (strcmp(schedule_name, "static") == 0) {
        #pragma omp parallel for num_threads(num_threads) schedule(static)
        for (int i = 1; i <= n; i++) {
            double result = compute_function(i);
            std::cout << "Thread " << omp_get_thread_num()
                << " processed i = " << i << std::endl;
        }
    } else if (strcmp(schedule_name, "dynamic") == 0) {
        #pragma omp parallel for num_threads(num_threads) schedule(dynamic, 1)
        for (int i = 1; i <= n; i++) {
            double result = compute_function(i);
            std::cout << "Thread " << omp_get_thread_num()
                << " processed i = " << i << std::endl;
        }
    } else if (strcmp(schedule_name, "guided") == 0) {
        #pragma omp parallel for num_threads(num_threads) schedule(guided)
        for (int i = 1; i <= n; i++) {
            double result = compute_function(i);
            std::cout << "Thread " << omp_get_thread_num()
                << " processed i = " << i << std::endl;
        }
    }
}

```

```

auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Time for " << schedule_name << " schedule: "
    << duration.count() << " ms" << std::endl;
}

int main() {
    const int n = 20;
    const int num_threads = 4;

    std::cout << "Comparing scheduling strategies with " << num_threads
        << " threads and " << n << " iterations" << std::endl;

    test_schedule("static", n, num_threads);
    test_schedule("dynamic", n, num_threads);
    test_schedule("guided", n, num_threads);

    return 0;
}

```

## 3.2 Static Scheduling with Different Chunk Sizes

### Example 3.2: Static Schedule Variations

cpp

```

#include <iostream>
#include <omp.h>

void demonstrate_static_scheduling() {
    const int n = 12;
    const int num_threads = 3;

    std::cout << "=== Static Schedule (default chunk) ===" << std::endl;
    #pragma omp parallel for num_threads(num_threads) schedule(static)
    for (int i = 0; i < n; i++) {
        std::cout << "Thread " << omp_get_thread_num()
                  << " processes iteration " << i << std::endl;
    }

    std::cout << "\n=== Static Schedule (chunk=2) ===" << std::endl;
    #pragma omp parallel for num_threads(num_threads) schedule(static, 2)
    for (int i = 0; i < n; i++) {
        std::cout << "Thread " << omp_get_thread_num()
                  << " processes iteration " << i << std::endl;
    }

    std::cout << "\n=== Static Schedule (chunk=1) ===" << std::endl;
    #pragma omp parallel for num_threads(num_threads) schedule(static, 1)
    for (int i = 0; i < n; i++) {
        std::cout << "Thread " << omp_get_thread_num()
                  << " processes iteration " << i << std::endl;
    }
}

int main() {
    demonstrate_static_scheduling();
    return 0;
}

```

### Exercise 3.1:

1. Run the scheduling comparison example
2. Observe which schedule performs best for unbalanced workloads
3. Modify the chunk size for static and dynamic schedules
4. Explain when each scheduling strategy would be most appropriate

## Part 4: The Single Directive

### 4.1 Understanding the Single Directive

The `single` directive specifies that the associated block is executed by only one thread (not necessarily the master thread). All other threads wait at an implicit barrier.

#### Syntax:

```
cpp
#pragma omp single [clauses]
structured block
```

#### Example 4.1: Basic Single Directive

```
cpp
```

```

#include <iostream>
#include <omp.h>

int main() {
    std::cout << "Before parallel region" << std::endl;

    #pragma omp parallel num_threads(4)
    {
        std::cout << "Thread " << omp_get_thread_num()
            << " before single region" << std::endl;

        #pragma omp single
        {
            std::cout << "Only thread " << omp_get_thread_num()
                << " executes this single block" << std::endl;
            // Simulate some work
            for (int i = 0; i < 3; i++) {
                std::cout << "Single thread working... step " << i+1 << std::endl;
            }
        }

        std::cout << "Thread " << omp_get_thread_num()
            << " after single region" << std::endl;
    }

    std::cout << "After parallel region" << std::endl;
    return 0;
}

```

## 4.2 Single Directive with Copyprivate

The `copyprivate` clause broadcasts a value from the executing thread to all other threads.

### Example 4.2: Single with Copyprivate

cpp

```

#include <iostream>
#include <omp.h>

int main() {
    int x = 5;

    std::cout << "Initial x = " << x << std::endl;

    #pragma omp parallel num_threads(4) default(none) private(x)
    {
        std::cout << "Thread " << omp_get_thread_num()
            << " before single: x = " << x << " (uninitialized)" << std::endl;

        #pragma omp single copyprivate(x)
        {
            x = 42;
            std::cout << "Thread " << omp_get_thread_num()
                << " in single block set x = " << x << std::endl;
        }

        std::cout << "Thread " << omp_get_thread_num()
            << " after single: x = " << x << std::endl;
    }

    std::cout << "After parallel region: x = " << x << std::endl;
    return 0;
}

```

### Exercise 4.1:

1. Run both single directive examples
2. Observe which thread executes the single block (run multiple times)
3. Explain the purpose of the copyprivate clause
4. Modify the copyprivate example to broadcast multiple variables

## Part 5: Tutorial 2 - Bubble Sort

### 5.1 Serial Bubble Sort

#### Example 5.1: Serial Bubble Sort Implementation





```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

void bubble_sort_serial(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

void print_array(const std::vector<int>& arr, const std::string& label) {
    std::cout << label << ": ";
    for (size_t i = 0; i < std::min((size_t)20, arr.size()); i++) {
        std::cout << arr[i] << " ";
    }
    if (arr.size() > 20) std::cout << "...";
    std::cout << std::endl;
}

int main() {
    const int n = 10000;
    std::vector<int> arr(n);

    // Initialize with random values
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 1000);

    for (int& val : arr) {
        val = dis(gen);
    }

    print_array(arr, "Original array");

    auto start = std::chrono::high_resolution_clock::now();
    bubble_sort_serial(arr);

```

```
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

print_array(arr, "Sorted array");
std::cout << "Serial bubble sort time: " << duration.count() << " ms" << std::endl;

// Verify sorting
bool is_sorted = std::is_sorted(arr.begin(), arr.end());
std::cout << "Array is sorted: " << (is_sorted ? "Yes" : "No") << std::endl;

return 0;
}
```

## 5.2 Odd-Even Transposition Sort (Parallel Alternative)

Since bubble sort has inherent dependencies, we use odd-even transposition sort which can be parallelized.

### Example 5.2: Serial Odd-Even Transposition Sort

cpp

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

void odd_even_sort_serial(std::vector<int>& arr) {
    int n = arr.size();
    bool sorted = false;

    while (!sorted) {
        sorted = true;

        // Odd phase
        for (int i = 1; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                std::swap(arr[i], arr[i + 1]);
                sorted = false;
            }
        }

        // Even phase
        for (int i = 0; i < n - 1; i += 2) {
            if (arr[i] > arr[i + 1]) {
                std::swap(arr[i], arr[i + 1]);
                sorted = false;
            }
        }
    }
}

int main() {
    const int n = 10000;
    std::vector<int> arr(n);

    // Initialize with random values
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 1000);

    for (int& val : arr) {
        val = dis(gen);
    }
}
```

```

std::cout << "Array size: " << n << std::endl;

auto start = std::chrono::high_resolution_clock::now();
odd_even_sort_serial(arr);
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Serial odd-even sort time: " << duration.count() << " ms" << std::endl;

// Verify sorting
bool is_sorted = std::is_sorted(arr.begin(), arr.end());
std::cout << "Array is sorted: " << (is_sorted ? "Yes" : "No") << std::endl;

return 0;
}

```

## 5.3 Parallel Odd-Even Transposition Sort

### Example 5.3: First Parallel Implementation

```
cpp
```

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <omp.h>

void odd_even_sort_parallel_v1(std::vector<int>& arr, int num_threads) {
    int n = arr.size();

    for (int phase = 0; phase < n; phase++) {
        if (phase % 2 == 1) {
            // Odd phase - compare odd indexed elements with their right neighbors
            #pragma omp parallel for num_threads(num_threads)
            for (int i = 1; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    std::swap(arr[i], arr[i + 1]);
                }
            }
        } else {
            // Even phase - compare even indexed elements with their right neighbors
            #pragma omp parallel for num_threads(num_threads)
            for (int i = 0; i < n - 1; i += 2) {
                if (arr[i] > arr[i + 1]) {
                    std::swap(arr[i], arr[i + 1]);
                }
            }
        }
    }
}

int main() {
    const int n = 10000;
    const int num_threads = 4;
    std::vector<int> arr(n);

    // Initialize with random values
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 1000);

    for (int& val : arr) {
        val = dis(gen);
    }
}

```

```

}

std::cout << "Array size: " << n << ", Threads: " << num_threads << std::endl;

auto start = std::chrono::high_resolution_clock::now();
odd_even_sort_parallel_v1(arr, num_threads);
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Parallel odd-even sort (v1) time: " << duration.count() << " ms" << std::endl;

// Verify sorting
bool is_sorted = std::is_sorted(arr.begin(), arr.end());
std::cout << "Array is sorted: " << (is_sorted ? "Yes" : "No") << std::endl;

return 0;
}

```

## 5.4 Optimized Parallel Implementation

### Example 5.4: Second Parallel Implementation with Combined Directives

cpp

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <omp.h>

void odd_even_sort_parallel_v2(std::vector<int>& arr, int num_threads) {
    int n = arr.size();

    #pragma omp parallel num_threads(num_threads) default(none) shared(arr, n)
    {
        for (int phase = 0; phase < n; phase++) {
            if (phase % 2 == 1) {
                // Odd phase
                #pragma omp for
                for (int i = 1; i < n - 1; i += 2) {
                    if (arr[i] > arr[i + 1]) {
                        std::swap(arr[i], arr[i + 1]);
                    }
                }
            }
            else {
                // Even phase
                #pragma omp for
                for (int i = 0; i < n - 1; i += 2) {
                    if (arr[i] > arr[i + 1]) {
                        std::swap(arr[i], arr[i + 1]);
                    }
                }
            }
        }
    }
}

int main() {
    const int n = 10000;
    const int num_threads = 4;
    std::vector<int> arr(n);

    // Initialize with random values
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 1000);

```



```

for (int& val : arr) {
    val = dis(gen);
}

std::cout << "Array size: " << n << ", Threads: " << num_threads << std::endl;

auto start = std::chrono::high_resolution_clock::now();
odd_even_sort_parallel_v2(arr, num_threads);
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Parallel odd-even sort (v2) time: " << duration.count() << " ms" << std::endl;

// Verify sorting
bool is_sorted = std::is_sorted(arr.begin(), arr.end());
std::cout << "Array is sorted: " << (is_sorted ? "Yes" : "No") << std::endl;

return 0;
}

```

### Exercise 5.1:

1. Implement and compare all three sorting algorithms
2. Test with different array sizes (1000, 10000, 50000)
3. Vary the number of threads (2, 4, 8) and measure performance
4. Explain why odd-even sort can be parallelized while bubble sort cannot

## Part 6: Tutorial 3 - Matrix-Vector Multiplication

### 6.1 Serial Matrix-Vector Multiplication

#### Example 6.1: Serial Implementation

cpp

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <iomanip>

void matrix_vector_multiply_serial(const std::vector<std::vector<double>>& matrix,
                                   const std::vector<double>& vector,
                                   std::vector<double>& result) {
    int m = matrix.size(); // number of rows
    int n = matrix[0].size(); // number of columns

    for (int i = 0; i < m; i++) {
        result[i] = 0.0;
        for (int j = 0; j < n; j++) {
            result[i] += matrix[i][j] * vector[j];
        }
    }
}

void print_matrix(const std::vector<std::vector<double>>& matrix, const std::string& name) {
    std::cout << name << " (showing first 5x5):" << std::endl;
    for (size_t i = 0; i < std::min((size_t)5, matrix.size()); i++) {
        for (size_t j = 0; j < std::min((size_t)5, matrix[i].size()); j++) {
            std::cout << std::setw(8) << std::fixed << std::setprecision(2) << matrix[i][j] << " ";
        }
        std::cout << std::endl;
    }
    std::cout << std::endl;
}

void print_vector(const std::vector<double>& vec, const std::string& name) {
    std::cout << name << " (showing first 10 elements): ";
    for (size_t i = 0; i < std::min((size_t)10, vec.size()); i++) {
        std::cout << std::setw(6) << std::fixed << std::setprecision(2) << vec[i] << " ";
    }
    std::cout << std::endl << std::endl;
}

int main() {
    const int m = 2000; // matrix rows
    const int n = 2000; // matrix columns (and vector size)

```

```

// Initialize matrix and vector with random values
std::random_device rd;
std::mt19937 gen(rd());
std::uniform_real_distribution<> dis(0.0, 10.0);

std::vector<std::vector<double>> matrix(m, std::vector<double>(n));
std::vector<double> vector(n);
std::vector<double> result(m);

// Fill matrix
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        matrix[i][j] = dis(gen);
    }
}

// Fill vector
for (int i = 0; i < n; i++) {
    vector[i] = dis(gen);
}

std::cout << "Matrix size: " << m << "x" << n << std::endl;
std::cout << "Vector size: " << n << std::endl << std::endl;

print_matrix(matrix, "Matrix A");
print_vector(vector, "Vector x");

auto start = std::chrono::high_resolution_clock::now();
matrix_vector_multiply_serial(matrix, vector, result);
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

print_vector(result, "Result y = A*x");
std::cout << "Serial matrix-vector multiplication time: " << duration.count() << " ms" << std::endl;

return 0;
}

```

## 6.2 Parallel Matrix-Vector Multiplication - Method 1

### Example 6.2: Basic Parallel Implementation

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <iomanip>
#include <omp.h>

void matrix_vector_multiply_parallel_v1(const std::vector<std::vector<double>>& matrix,
                                       const std::vector<double>& vector,
                                       std::vector<double>& result,
                                       int num_threads) {
    int m = matrix.size(); // number of rows
    int n = matrix[0].size(); // number of columns

    #pragma omp parallel for num_threads(num_threads) default(none) shared(matrix, vector, result, m, n)
    for (int i = 0; i < m; i++) {
        result[i] = 0.0;
        for (int j = 0; j < n; j++) {
            result[i] += matrix[i][j] * vector[j];
        }

        // Show thread work distribution for small matrices
        if (m <= 20) {
            std::cout << "Thread " << omp_get_thread_num()
                      << " computed row " << i << std::endl;
        }
    }
}

int main() {
    const int m = 2000;
    const int n = 2000;
    const int num_threads = 4;

    // Initialize matrix and vector
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(0.0, 10.0);

    std::vector<std::vector<double>> matrix(m, std::vector<double>(n));
    std::vector<double> vector(n);
    std::vector<double> result(m);

```

```

// Fill matrix and vector
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        matrix[i][j] = dis(gen);
    }
}

for (int i = 0; i < n; i++) {
    vector[i] = dis(gen);
}

std::cout << "Matrix size: " << m << "x" << n << ", Threads: " << num_threads << std::endl;

auto start = std::chrono::high_resolution_clock::now();
matrix_vector_multiply_parallel_v1(matrix, vector, result, num_threads);
auto end = std::chrono::high_resolution_clock::now();

auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Parallel matrix-vector multiplication (v1) time: "
    << duration.count() << " ms" << std::endl;

return 0;
}

```

## 6.3 Parallel Matrix-Vector Multiplication - Method 2 (with Scheduling)

### Example 6.3: Parallel Implementation with Different Scheduling

cpp

```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <omp.h>

void matrix_vector_multiply_scheduled(const std::vector<std::vector<double>>& matrix,
                                     const std::vector<double>& vector,
                                     std::vector<double>& result,
                                     int num_threads,
                                     const std::string& schedule_type) {
    int m = matrix.size();
    int n = matrix[0].size();

    if (schedule_type == "static") {
        #pragma omp parallel for num_threads(num_threads) schedule(static) default(none) shared(matrix, vector, result, n)
        for (int i = 0; i < m; i++) {
            result[i] = 0.0;
            for (int j = 0; j < n; j++) {
                result[i] += matrix[i][j] * vector[j];
            }
        }
    } else if (schedule_type == "dynamic") {
        #pragma omp parallel for num_threads(num_threads) schedule(dynamic, 10) default(none) shared(matrix, vector, result, n)
        for (int i = 0; i < m; i++) {
            result[i] = 0.0;
            for (int j = 0; j < n; j++) {
                result[i] += matrix[i][j] * vector[j];
            }
        }
    } else if (schedule_type == "guided") {
        #pragma omp parallel for num_threads(num_threads) schedule(guided) default(none) shared(matrix, vector, result, n)
        for (int i = 0; i < m; i++) {
            result[i] = 0.0;
            for (int j = 0; j < n; j++) {
                result[i] += matrix[i][j] * vector[j];
            }
        }
    }
}

void test_scheduling(const std::vector<std::vector<double>>& matrix,
                    const std::vector<double>& vector,

```

```

        int num_threads) {
std::vector<std::string> schedules = {"static", "dynamic", "guided"};

for (const auto& schedule : schedules) {
    std::vector<double> result(matrix.size());

    auto start = std::chrono::high_resolution_clock::now();
    matrix_vector_multiply_scheduled(matrix, vector, result, num_threads, schedule);
    auto end = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Schedule " << schedule << ": " << duration.count() << " ms" << std::endl;
}
}

int main() {
    const int m = 2000;
    const int n = 2000;
    const int num_threads = 4;

    // Initialize matrix and vector
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(0.0, 10.0);

    std::vector<std::vector<double>> matrix(m, std::vector<double>(n));
    std::vector<double> vector(n);

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            matrix[i][j] = dis(gen);
        }
    }

    for (int i = 0; i < n; i++) {
        vector[i] = dis(gen);
    }

    std::cout << "Matrix size: " << m << "x" << n << ", Threads: " << num_threads << std::endl;
    std::cout << "Comparing different scheduling strategies:" << std::endl;

    test_scheduling(matrix, vector, num_threads);

```

```
    return 0;  
}
```

## 6.4 Performance Analysis Program

### Example 6.4: Comprehensive Performance Analysis

```
cpp
```



```

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <omp.h>
#include <iomanip>

void matrix_vector_multiply_serial(const std::vector<std::vector<double>>& matrix,
                                   const std::vector<double>& vector,
                                   std::vector<double>& result) {
    int m = matrix.size();
    int n = matrix[0].size();

    for (int i = 0; i < m; i++) {
        result[i] = 0.0;
        for (int j = 0; j < n; j++) {
            result[i] += matrix[i][j] * vector[j];
        }
    }
}

void matrix_vector_multiply_parallel(const std::vector<std::vector<double>>& matrix,
                                    const std::vector<double>& vector,
                                    std::vector<double>& result,
                                    int num_threads) {
    int m = matrix.size();
    int n = matrix[0].size();

    #pragma omp parallel for num_threads(num_threads) default(none) shared(matrix, vector, result, m, n)
    for (int i = 0; i < m; i++) {
        result[i] = 0.0;
        for (int j = 0; j < n; j++) {
            result[i] += matrix[i][j] * vector[j];
        }
    }
}

int main() {
    const std::vector<int> matrix_sizes = {500, 1000, 2000};
    const std::vector<int> thread_counts = {1, 2, 4, 8};

    std::cout << std::setw(12) << "Matrix Size"
              << std::setw(10) << "Threads"

```

```

    << std::setw(15) << "Time (ms)"
    << std::setw(12) << "Speedup"
    << std::setw(12) << "Efficiency" << std::endl;
std::cout << std::string(61, '-') << std::endl;

for (int size : matrix_sizes) {
    // Initialize matrix and vector
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<> dis(0.0, 10.0);

    std::vector<std::vector<double>> matrix(size, std::vector<double>(size));
    std::vector<double> vector(size);

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            matrix[i][j] = dis(gen);
        }
        vector[i] = dis(gen);
    }

    double serial_time = 0.0;

    for (int threads : thread_counts) {
        std::vector<double> result(size);

        auto start = std::chrono::high_resolution_clock::now();

        if (threads == 1) {
            matrix_vector_multiply_serial(matrix, vector, result);
        } else {
            matrix_vector_multiply_parallel(matrix, vector, result, threads);
        }

        auto end = std::chrono::high_resolution_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
        double time_ms = duration.count();

        if (threads == 1) {
            serial_time = time_ms;
        }

        double speedup = (threads == 1) ? 1.0 : serial_time / time_ms;
        double efficiency = speedup / threads;
    }
}

```

```

std::cout << std::setw(12) << size << "x" << size
    << std::setw(10) << threads
    << std::setw(15) << std::fixed << std::setprecision(2) << time_ms
    << std::setw(12) << std::fixed << std::setprecision(2) << speedup
    << std::setw(12) << std::fixed << std::setprecision(2) << efficiency
    << std::endl;
}
std::cout << std::string(61, '-') << std::endl;
}

return 0;
}

```

### Exercise 6.1:

1. Implement all three versions of matrix-vector multiplication
2. Run the performance analysis and record the results
3. Test with different scheduling strategies and compare performance
4. Explain why this problem parallelizes well
5. Calculate the theoretical vs actual speedup for different thread counts

## Part 7: Advanced Synchronization

### 7.1 The Barrier Directive

#### Example 7.1: Using Barriers for Synchronization

cpp

```

#include <iostream>
#include <omp.h>
#include <vector>

int main() {
    const int n = 8;
    std::vector<int> a(n), b(n);

    // Initialize array a
    for (int i = 0; i < n; i++) {
        a[i] = i * 2;
    }

    std::cout << "Initial array a: ";
    for (int val : a) std::cout << val << " ";
    std::cout << std::endl;

    #pragma omp parallel num_threads(4) default(none) shared(a, b, n)
    {
        int thread_id = omp_get_thread_num();
        int num_threads = omp_get_num_threads();

        // Phase 1: Each thread modifies its portion of array a
        int start = thread_id * n / num_threads;
        int end = (thread_id + 1) * n / num_threads;

        for (int i = start; i < end; i++) {
            a[i] = a[i] * 3;
            std::cout << "Thread " << thread_id << " modified a[" << i << "] = " << a[i] << std::endl;
        }

        // Barrier ensures all threads complete Phase 1 before Phase 2
        #pragma omp barrier

        // Phase 2: Each thread reads from neighbor's data
        int neighbor = (thread_id == 0) ? num_threads - 1 : thread_id - 1;
        int neighbor_start = neighbor * n / num_threads;
        int neighbor_end = (neighbor + 1) * n / num_threads;

        for (int i = start; i < end; i++) {
            int neighbor_idx = neighbor_start + (i - start) % (neighbor_end - neighbor_start);
            b[i] = a[neighbor_idx] + 100;
            std::cout << "Thread " << thread_id << " computed b[" << i << "] = " << b[i]

```

```

        << " (using a[" << neighbor_idx << "])" << std::endl;
    }
}

std::cout << "Final array a: ";
for (int val : a) std::cout << val << " ";
std::cout << std::endl;

std::cout << "Final array b: ";
for (int val : b) std::cout << val << " ";
std::cout << std::endl;

return 0;
}

```

## 7.2 Atomic Operations

### Example 7.2: Comparing Critical vs Atomic

cpp

```

#include <iostream>
#include <omp.h>
#include <chrono>

int main() {
    const int n = 10000000;
    int counter_critical = 0;
    int counter_atomic = 0;

    // Method 1: Using critical section
    auto start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel for num_threads(4)
    for (int i = 0; i < n; i++) {
        #pragma omp critical
        counter_critical++;
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto critical_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    // Method 2: Using atomic operation
    start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel for num_threads(4)
    for (int i = 0; i < n; i++) {
        #pragma omp atomic
        counter_atomic++;
    }

    end = std::chrono::high_resolution_clock::now();
    auto atomic_time = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Counter with critical section: " << counter_critical << std::endl;
    std::cout << "Counter with atomic operation: " << counter_atomic << std::endl;
    std::cout << "Critical section time: " << critical_time.count() << " ms" << std::endl;
    std::cout << "Atomic operation time: " << atomic_time.count() << " ms" << std::endl;
    std::cout << "Speedup using atomic: " << (double)critical_time.count() / atomic_time.count() << "x" << std::endl;

    return 0;
}

```

## Exercise 7.1:

1. Run the barrier example and understand the synchronization points
  2. Compare critical section vs atomic performance
  3. Modify the atomic example to use different atomic operations (+, \*, etc.)
- 

## Lab Exercises

### Exercise 1: Work Sharing Fundamentals

Write a program that:

1. Creates an array of 1000 elements initialized with their index values
2. Uses `#pragma omp for` to square each element in parallel
3. Prints which thread processes which iterations (for first 20 iterations)
4. Compares performance with serial version

**Constraint:** Use `#pragma omp parallel` and `#pragma omp for` separately, then compare with `#pragma omp parallel for`.

### Exercise 2: Loop Dependency Analysis

Given the following loops, determine which ones can be parallelized and explain why:

```
cpp
// Loop A
for (int i = 1; i < n; i++) {
    a[i] = a[i-1] + b[i];
}

// Loop B
for (int i = 0; i < n; i++) {
    a[i] = b[i] * c[i];
}

// Loop C
for (int i = 0; i < n-1; i++) {
    a[i] = (b[i] + b[i+1]) / 2;
}
```

**Constraint:** Provide detailed analysis of dependencies and implement parallel versions where possible.

### Exercise 3: $\pi$ Estimation Comparison

Implement three versions of  $\pi$  estimation:

1. Serial version
2. Parallel with critical section
3. Parallel with reduction
4. Test with different numbers of intervals ( $10^6$ ,  $10^7$ ,  $10^8$ )
5. Record timing and accuracy results

**Constraint:** Use exactly the mathematical formula provided in the tutorial.

## Exercise 4: Scheduling Strategy Analysis

Create a program with artificial load imbalance:

1. Implement a function `work(int i)` that takes `i*1000` iterations to complete
2. Parallelize a loop calling this function with different scheduling strategies
3. Measure and compare performance of static, dynamic, and guided scheduling
4. Test with different chunk sizes

**Constraint:** Use exactly 4 threads and loop from  $i=1$  to  $i=20$ .

## Exercise 5: Single Directive Application

Write a program that:

1. Reads matrix dimensions from user input (use single directive)
2. Initializes a matrix with random values in parallel
3. Computes row sums in parallel
4. Finds the maximum row sum using single directive with `copyprivate`

**Constraint:** Use `#pragma omp single copyprivate` to broadcast the maximum value.

## Exercise 6: Odd-Even Sort Implementation

Implement and compare:

1. Serial bubble sort
2. Serial odd-even transposition sort
3. Parallel odd-even transposition sort (both versions from tutorial)
4. Test with arrays of size 1000, 5000, 10000



5. Verify correctness and measure performance

**Constraint:** Show the number of phases required for sorting completion.

**Exercise 7: Matrix Operations Suite**

Create a comprehensive matrix operations program:

- 1. Matrix-vector multiplication (from tutorial)
- 2. Matrix-matrix multiplication (new implementation)
- 3. Matrix transpose operation
- 4. Compare serial vs parallel performance for different matrix sizes
- 5. Test different scheduling strategies

**Constraint:** Use different scheduling for each operation and justify your choices.

**Exercise 8: Advanced Synchronization**

Implement a producer-consumer-like scenario:

- 1. Multiple threads generate data and store in a shared array
- 2. Use barriers to synchronize phases of computation
- 3. One thread processes the accumulated data using `single` directive
- 4. Compare performance with and without proper synchronization

**Constraint:** Use `#pragma omp barrier` and demonstrate its necessity.

**Performance Analysis Template**

For timing exercises, record results in this format:

| Problem Size | Threads | Schedule | Time (ms) | Speedup | Efficiency |
|--------------|---------|----------|-----------|---------|------------|
| 1000         | 1       | serial   | X         | 1.0     | 1.0        |
| 1000         | 2       | static   | Y         | X/Y     | (X/Y)/2    |
| 1000         | 4       | static   | Z         | X/Z     | (X/Z)/4    |

**Analysis Questions:**

- 1. Which scheduling strategy performs best for your workload?
- 2. How does performance scale with the number of threads?

3. What factors limit the achievable speedup?
4. When does the overhead of parallelization outweigh the benefits?

## Compilation and Execution

### Compilation:

```
bash  
g++ -g -Wall -fopenmp -o program program.cpp
```

### Setting Thread Count:

```
bash  
export OMP_NUM_THREADS=4  
./program
```

### For timing accuracy:

```
bash  
time ./program
```

## Submission Guidelines

1. Submit all source code files with detailed comments
2. Include performance analysis tables for timing exercises
3. Provide explanations for dependency analysis exercises
4. Document your findings about different scheduling strategies
5. Include sample outputs demonstrating correctness
6. Answer all analysis questions in your report

## Additional Resources

- OpenMP 4.5 Specification: <https://www.openmp.org/wp-content/uploads/openmp-4.5.pdf>
- Intel OpenMP Reference:  
<https://software.intel.com/content/www/us/en/develop/documentation/cpp-compiler-developer-guide-and-reference/top/optimization-and-programming-guide/openmp-support.html>
- LLNL OpenMP Tutorial: <https://computing.llnl.gov/tutorials/openMP/>

